Computer Vision
Lecture 11: Video & Motion
Optical Flow, ID Tracking, Action Recognition

Nguyen Manh Toan

Swinburne Vietnam

Week 11

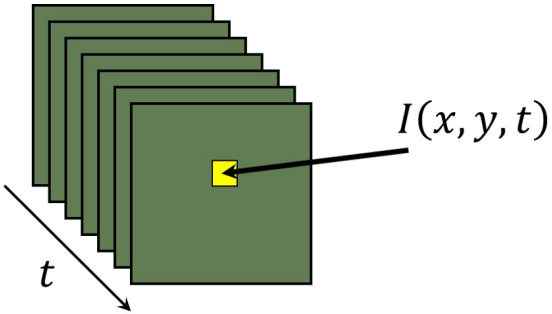
Where We Left Off

Lectures 7–10

One image at a time: boxes, masks, keypoints, or embeddings. The interesting decision was the **output space**, not the architecture.

Today: one more axis

A video is not a harder image; it is a sequence. A sequence asks three different questions that want three different answers.



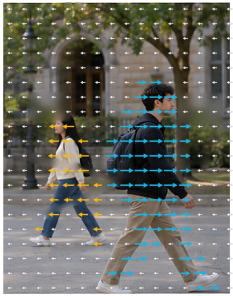
Both halves of this lecture contain a benchmark number that is famous, and **measures something other than what people think**. Lecture 9 warned you about mIoU; today it is MOTA, and Kinetics.

Three Questions a Video Asks

five frames

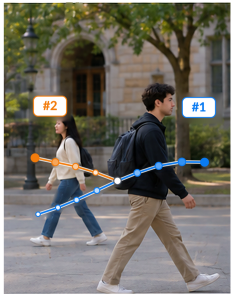


where did it move?



optical flow: a vector per pixel

which one is which?



tracking: an identity over time

what is happening?



"one object passes in front of another"

a label for the whole clip

action recognition

Adding a time axis creates three core video-understanding tasks: motion estimation, identity tracking, and action recognition.

Today's Lecture

Motion as a field

1. One equation, two unknowns
2. Lucas–Kanade, Horn–Schunck
3. FlowNet, PWC-Net, RAFT
4. Measuring flow, and what it was trained on

Identity over time

6. Tracking-by-detection
7. Bipartite matching, again
8. Kalman, SORT, ByteTrack
9. When the camera moves; SAM 2
10. Why MOTA is broken

What is happening

11. Why pooling cannot work
12. Two-stream, I3D, SlowFast
13. Video transformers
14. What time costs, and what Kinetics measures

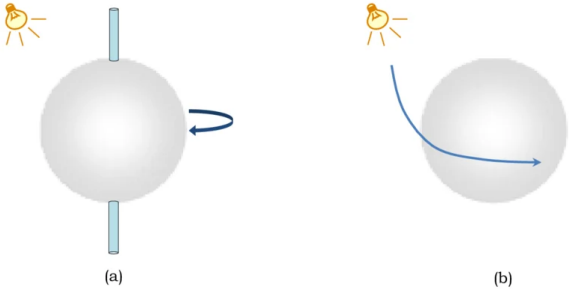
The three sections are roughly independent. If you only take one thing from each: **the aperture problem**, **HOTA**, and **the time-reversal test**.

Motion as a Dense Field

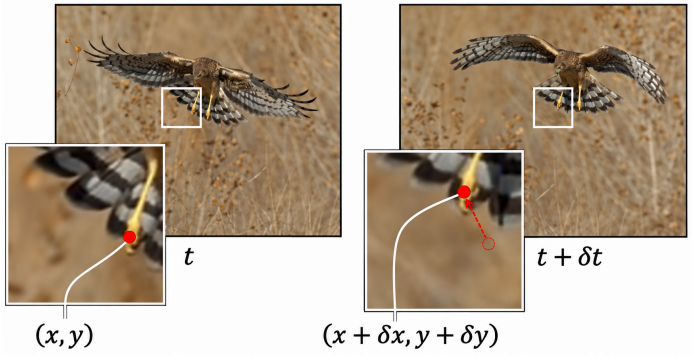
Optical Flow Is Not Motion

Optical flow is the **apparent** motion of brightness patterns, **not the true 3D physical motion** of objects in a scene

- A matte sphere rotating under fixed light has *zero* flow and plenty of motion.
- A stationary sphere under a moving light has plenty of flow and *zero* motion.
- A barber's pole moves horizontally and flows vertically.



Optical Flow



The assumption

Brightness constancy: a point (x, y) keeps its intensity as it moves.

$$I(x, y, t) = I(x + \delta x, y + \delta y, t + \delta t).$$

Optical Flow Equation

The assumption $I(x, y, t) = I(x + \delta x, y + \delta y, t + \delta t)$ leads to

$$I(x, y, t) \sim I(x, y, t) + I_x \delta x + I_y \delta y + I_t \delta t + \ldots \text{higher order terms} \quad \text{(Taylor expansion)},$$

where I_x, I_y are image gradient, I_t is the gradient along time (known).
Assume the displacement $(\delta x, \delta y)$ and δt are small, then $I_x \delta x + I_y \delta y + I_t \delta t = 0$.
Divide by δt leads to **Optical Flow equation**

$$\boxed{I_x u + I_y v + I_t = 0}$$

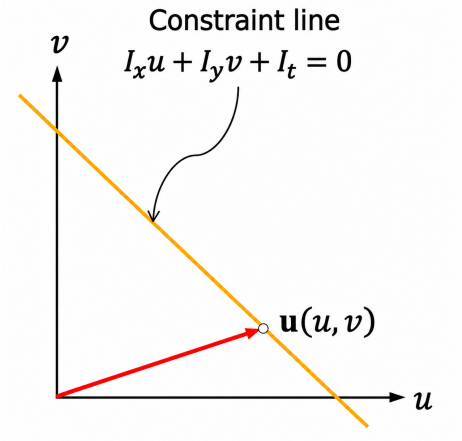
where $(u, v) = (\frac{dx}{dt}, \frac{dy}{dt})$ is the optical flow — the velocity vectors of the pixel movement along x and y (unknown).

Geometric Intepretation

For any point (x, y), the optical flow lies on the line

$$I_x u + I_y v + I_t = 0$$

A single equation with two unknowns suffers from the **aperture problem**



Normal Flow

Optical flow can be decomposed into

$$\mathbf{u} = \mathbf{u}_n + \mathbf{u}_p$$

where

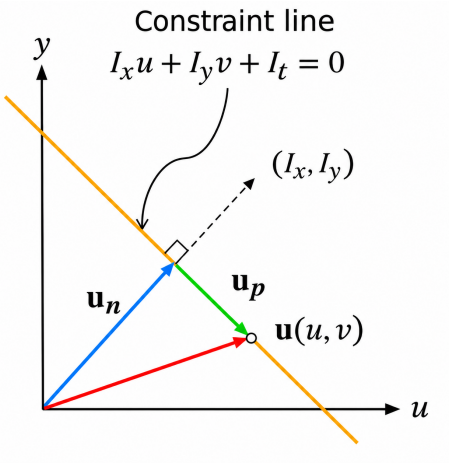
- $\mathbf{u}_n$: Normal Flow
- $\mathbf{u}_p$: Parallel Flow

Direction of Normal Flow

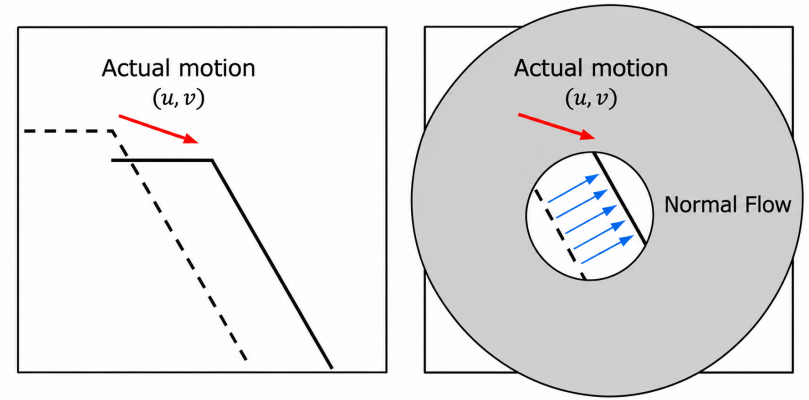
$$\widehat{\mathbf{u}}_n = \frac{(I_x, I_y)}{\sqrt{I_x^2 + I_y^2}}$$

Magnitude of Normal Flow

$$|\mathbf{u}_n| = \frac{|I_t|}{\sqrt{I_x^2 + I_y^2}}$$



Aperture Problem



To find the Optical Flow $\mathbf{u} = (u, v)$, different algorithms impose **additional constraints**.

Lucas–Kanade (1981)

Assume the flow is constant within a small window W, then for all $(i,j) \in W$

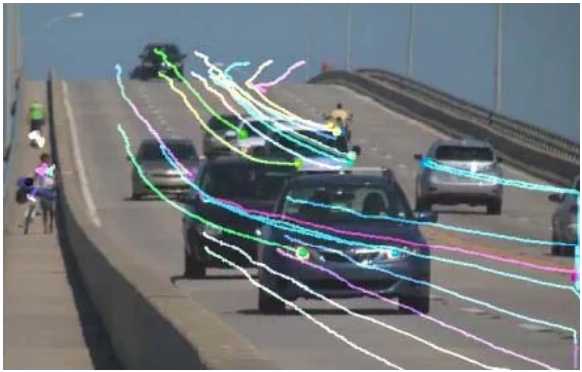
$$I_x(i,j)u + I_y(i,j)v + I_t = 0.$$

Stack the equations:

$$\begin{pmatrix} I_x(1,1) & I_y(1,1) \\ \vdots & \vdots \\ I_x(n,n) & I_y(n,n) \end{pmatrix} \begin{pmatrix} u \\ v \end{pmatrix} = - \begin{pmatrix} I_t(1,1) \\ \vdots \\ I_t(n,n) \end{pmatrix}$$

It has the form $A\mathbf{u} = B$, implying $A^T A\mathbf{u} = A^T B$. Solvable where that 2×2 matrix is well conditioned — i.e. at corners:

$$\mathbf{u} = (A^T A)^{-1} A^T B.$$



Sparse but reliable.

Horn–Schunck (1981): Regularization

Assume smoothness in the flow over the whole image.
Global energy functional:

$$E = \iint \left[\underbrace{(I_x u + I_y v + I_t)^2}_{\text{Brightness constancy}} + \lambda^2 \underbrace{\left(\|\nabla u\|^2 + \|\nabla v\|^2 \right)}_{\text{Smoothness Term}} \right] dx\, dy$$

The Euler-Lagrange Formulation

Minimizing the global energy functional yields a system of coupled partial differential equations (PDEs):

$$I_x(I_x u + I_y v + I_t) - \lambda^2 \nabla^2 u = 0$$
$$I_y(I_x u + I_y v + I_t) - \lambda^2 \nabla^2 v = 0$$

Where $\nabla^2 = \frac{\partial^2}{\partial x^2} + \frac{\partial^2}{\partial y^2}$ is the **Laplacian operator**, driving physical diffusion.

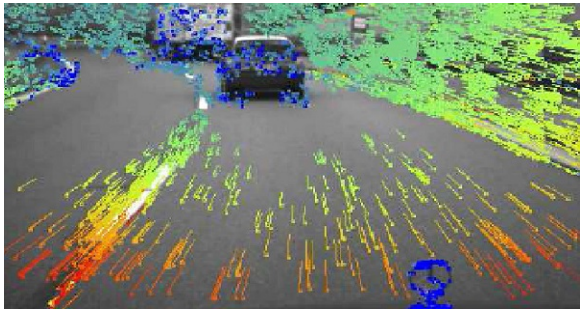
Iterative Resolution

Iterative Update Equations (Jacobi Method)

Approximating ∇^2 via neighborhood averages $(\bar{u}, \bar{v})$ decouples the system for step $k+1$:

$$u^{k+1} = \bar{u}^k - \frac{I_x(I_x\bar{u}^k + I_y\bar{v}^k + I_t)}{\lambda^2 + I_x^2 + I_y^2}$$

$$v^{k+1} = \bar{v}^k - \frac{I_y(I_x\bar{u}^k + I_y\bar{v}^k + I_t)}{\lambda^2 + I_x^2 + I_y^2}$$



Information flows from corners into flat regions: **Dense but soft.**

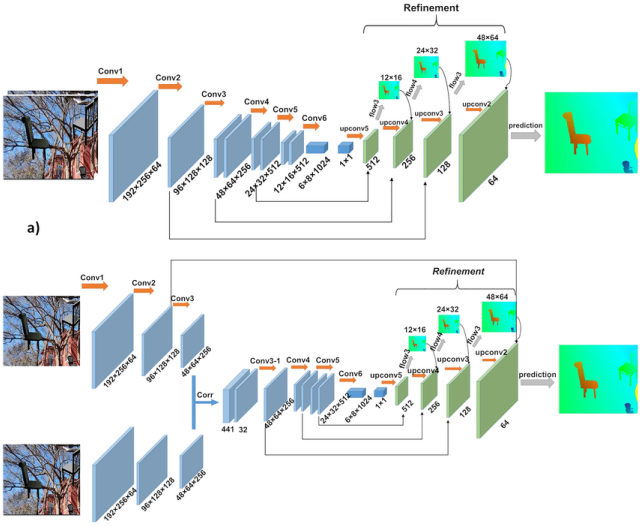
- **At Corners & Edges ($I_x, I_y \gg 0$):** Data terms dominate to anchor motion values.
- **In Flat Regions ($I_x, I_y \approx 0$):** Updates reduce to $u^{k+1} = \bar{u}^k$, meaning flow vectors purely **diffuse outward** into textureless zones like heat.

FlowNet (2015): Learn the Whole Thing

Two frames in, a flow field out, trained end to end. Two variants:

- **FlowNetS** stacks the two frames.
- **FlowNetC** encodes each frame separately, then computes an explicit *correlation* between them.

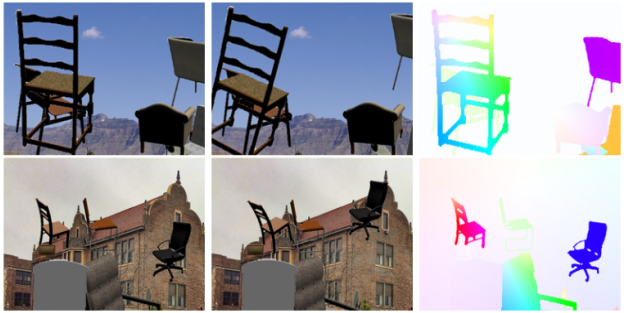
The second works better; the first hints that the classical structure was not the problem.



Data: The Unexpected Problem

A human can draw a box or a mask; no human can label the optical flow of every pixel.

Flow is trained on **synthetic** data — chairs flying over Flickr photographs, and later rendered films.

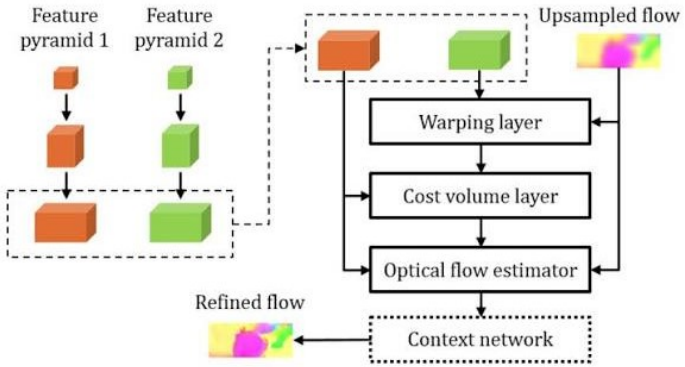


Supervised flow exists only because someone was willing to render the labels, and every benchmark number in this section inherits whatever that renderer got wrong.

PWC-Net (Sun et al., 2018): Put the Principles Back

Take the three things classical flow always did and build them into the network as layers:

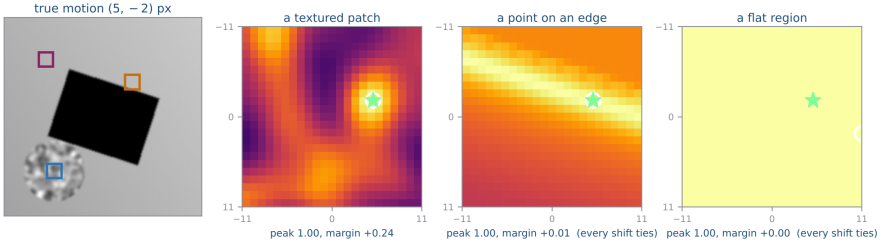
1. A **pyramid**: large motions are found coarsely first
2. **Warping** the second frame by the current estimate; each level only sees a small residual motion
3. A **cost volume** over a small displacement range



Hand-built structure → throw it away and learn → put the structure back *as architecture*. The prior was never the problem. Hard-coding it was.

The Cost Volume

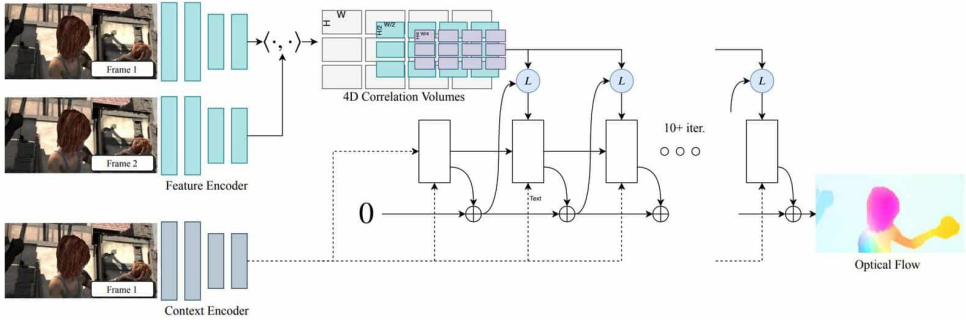
The Cost Volume is a key matching layer that measures the visual similarity between the features of the first image and the warped features of the second image across a local search window



A correlation volume is this picture computed at every pixel: for each location, the match score against every candidate displacement. The three cases are the aperture problem again, now visible as the shape of the cost surface --- a single sharp peak, a valley you can slide along, and nothing at all. RAFT's contribution was to build this for ALL pairs at once, then let a recurrent network look up from it repeatedly instead of trusting one argmax.

PWC-Net is compact, **17**× **smaller** than FlowNet2 and about **35 fps** at Sintel resolution, while beating every published method at the time on Sintel final and KITTI 2015.

RAFT (2020): All Pairs, Then Iterate



- per-pixel features;
- a **4-D correlation volume over all pairs of pixels** — every location against every location, at several scales;
- a **recurrent** unit looks up from that volume and updates the flow, dozens of times, at a single high resolution.

Why Dropping The Pyramid

A pyramid exists to find large motions. If the correlation volume *already* contains all displacements, no need to go coarse — and no longer inherit the pyramid's habit of losing small fast objects at the top level.

	Sintel final (EPE)	**KITTI (Fl-all)**
best published, before	4.098	6.10%
RAFT	2.855 (−30%)	5.10% (−16%)

Measuring Flow

Endpoint Error (EPE): mean Euclidean distance between predicted and true flow, in pixels.

$$\mathrm{EPE} = \frac{1}{N}\sum_{i=1}^{N}\sqrt{(u_{\mathrm{pred},i} - u_{\mathrm{true},i})^2 + (v_{\mathrm{pred},i} - v_{\mathrm{true},i})^2}$$

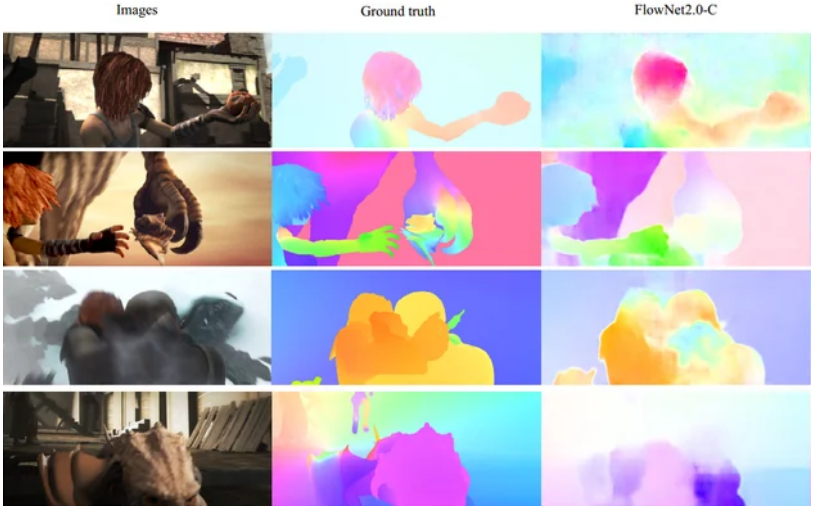
Simple, and dominated by the worst pixels.

Flow Incorrect - All Pixels (Fl-all): the percentage of pixels whose error exceeds 3 px *and* 5% of the true magnitude. An outlier rate, not an average.

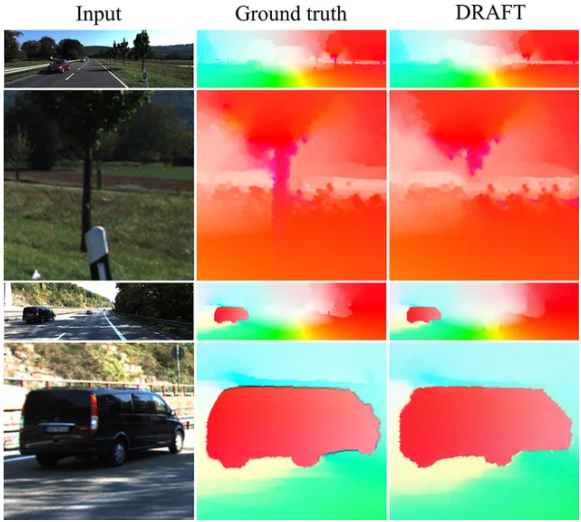
Where the ground truth came from

Sintel is a rendered film, so its labels are exact. KITTI is real driving, so its labels come from a laser scanner and are **sparse** — about 20% of pixels.

Sintel Dataset



KITTI Dataset



What "Trained On" Means

Dataset	What it actually is	Size
FlyingChairs	809 rendered chair types over 964 Flickr photos	22,872 pairs
FlyingThings3D	rendered objects on random 3-D trajectories	large
Sintel	frames from an open-source animated film	1,041 pairs per pass
KITTI	real driving; laser scanner, sparse labels	194 training pairs

The standard recipe

Pretrain on Chairs, then Things; finetune on a mixture of Sintel, KITTI and HD1K; then finetune on KITTI alone. Papers write this as $\mathbf{C} \rightarrow \mathbf{T} \rightarrow \mathbf{S}{+}\mathbf{K}{+}\mathbf{H} \rightarrow \mathbf{K}$.

Four stages, because no single dataset is both large enough and real enough.

What that is worth

RAFT after C+T, never having seen KITTI: **17.4%** Fl-all on the KITTI-15 training split. After the full schedule: **5.10%** on the KITTI test server.

Different splits, so not a controlled comparison — but a factor of three is not a split artefact.

So "RAFT gets 5.10% on KITTI" means *RAFT finetuned on KITTI*.

Where Flow Is Actually Used

Computed explicitly

- video compression and frame interpolation — here the motion *is* the output
- stabilisation, and camera-motion estimation inside trackers
- visual odometry and SLAM front-ends
- scientific imaging: particle velocimetry, cell migration, sea-ice and cloud drift in satellite imagery

Quietly disappeared

The two-stream era fed a precomputed flow field to a second network because the RGB network would not learn motion on its own. SlowFast's argument is that a high-frame-rate, low-capacity pathway does the same job from raw pixels; video transformers never had a flow stream at all; trackers predict with a Kalman filter.

The useful summary

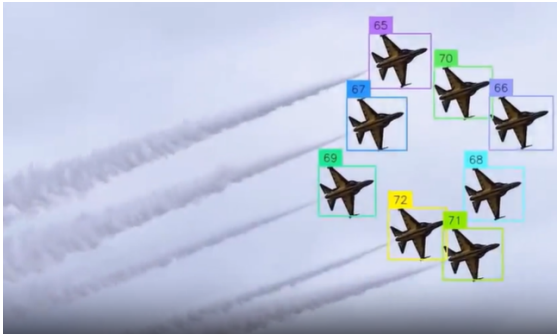
Flow is increasingly a *product* — the displacement field is what someone wanted — rather than an *ingredient* inside a recognition pipeline. Compute it when it is the output, not as a feature.

Identity Over Time

The Task Is Identity, Not Location

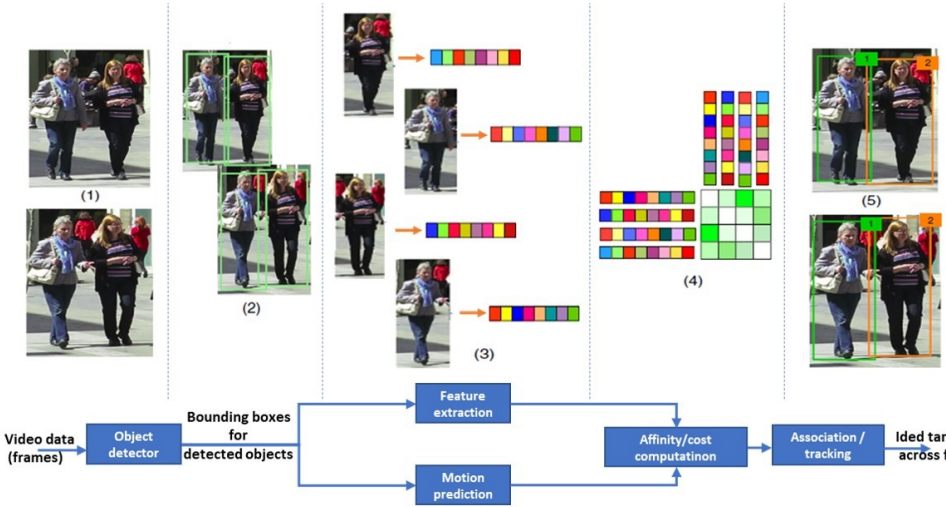
Running a detector on every frame, and having every object's location in every frame, is not tracking.

Tracking is the claim that *this* box and *that* box, one frame apart, are the same object — and keeping that claim alive through occlusion, crowding and missed detections.

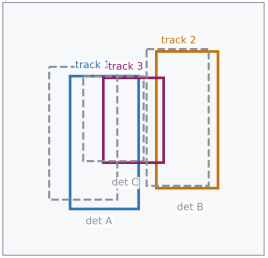


Single-object tracking: given the thing in frame 1 and must follow it. Everything today is **multi-object**.

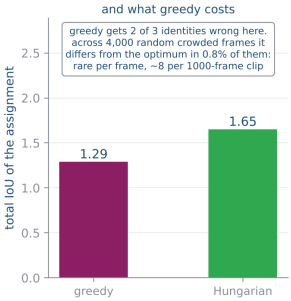
Tracking by Detection



Bipartite Matching, With t On The Other Axis

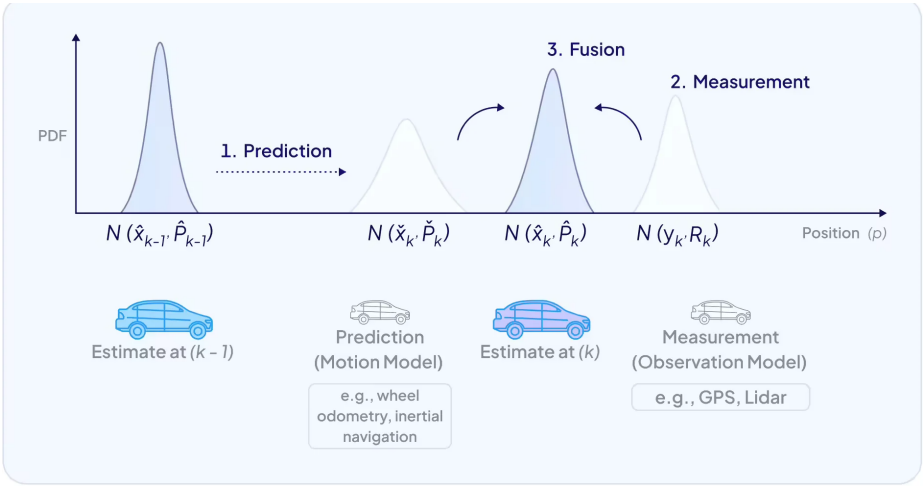




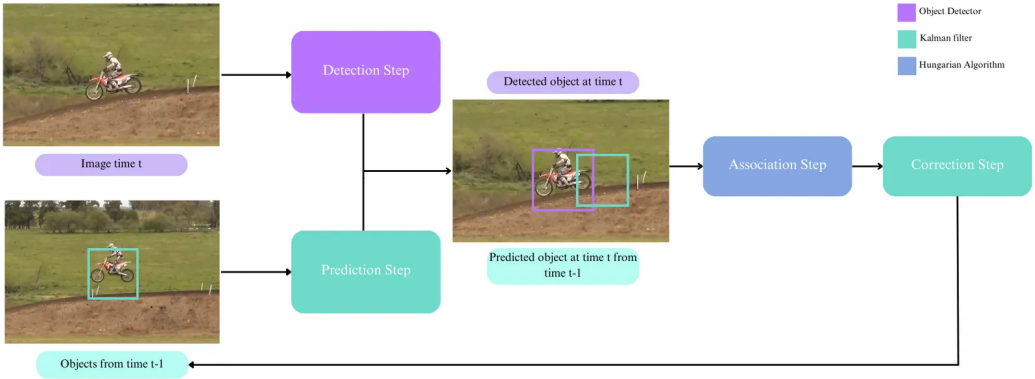


This is Lecture 8's figure with t on the other axis. There the Hungarian algorithm matched predictions to ground-truth objects inside one image; here it matches detections to tracks between two frames. Same cost matrix, same solver, and the same reason greedy is not good enough: taking the best pair first forecloses the pairs that were worth more together. Note how rarely it matters per frame, and how much it matters per sequence --- which is why this bug survived so long.

Kalman Filter: Prediction and Correction



SORT (2016): Simple Online Realtime Object Tracking



A Kalman filter with a constant-velocity model, IoU as the cost, and the Hungarian algorithm. No appearance, no learning, a few hundred lines.

SORT: How Little Is Enough

It was competitive with far heavier trackers, runs at 100s PFS — all the expensive work is in the detector, which we run anyway.

What it cannot do

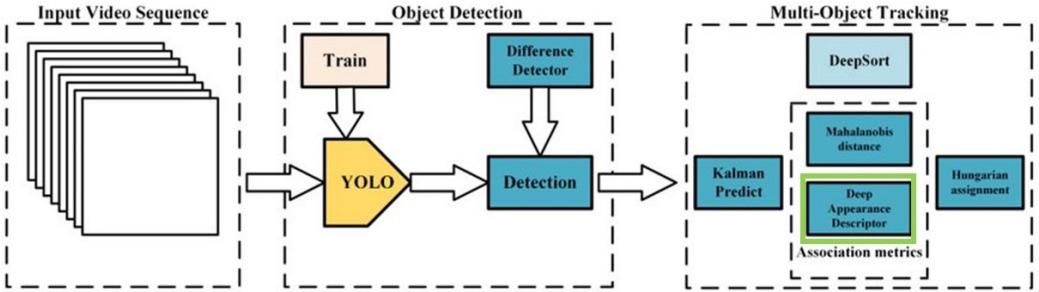
IoU is zero when two boxes do not overlap, so SORT loses anything occluded for more than a frame or two, and cannot re-acquire it afterwards.

Its identity switches are concentrated exactly where people cross.

The lesson, similar to SimpleBaseline's

Build the dullest thing that could work before you build the clever thing. SORT is still a reasonable first tracker for a student project, and it will tell you whether your problem is association at all — or just detection.

DeepSORT: Simple Online and Realtime Tracking with a Deep Association Metric



Add a second cost: the cosine distance between **appearance embeddings** of the detection and the track's recent history. Combine it with the motion cost, and gate both by a Mahalanobis distance so the appearance term can never override physics.

DeepSORT: Pros and Cons

Appearance embeddings is exactly Lecture 10's problem — an open set, trained with a margin loss, read with a cosine — applied to people instead of faces.

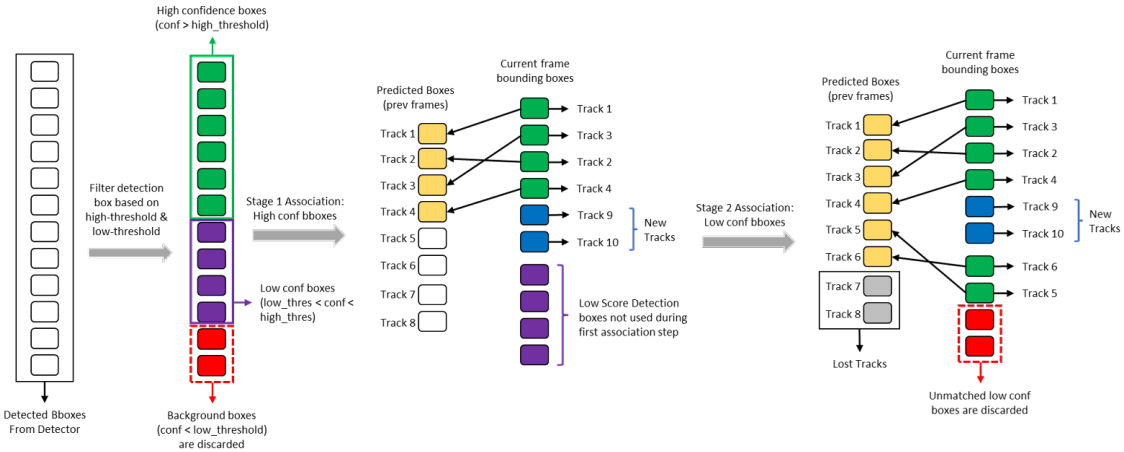
What it fixes, and what it costs

Re-acquisition after a long occlusion, which motion alone cannot do.

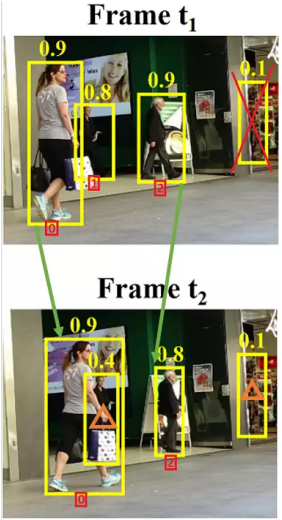
You now run a second network per detection, and inherit everything Lecture 10 said about embeddings: the threshold is yours to set, and the error rate is not the same for everyone in the scene.

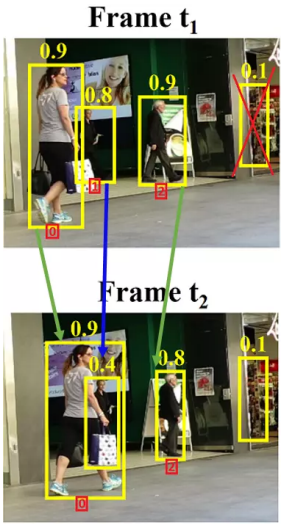
Later trackers fold the embedding into the detector so the crops are free (JDE, FairMOT). The association logic does not change.

ByteTrack (2022): Keep the Weak Boxes



ByteTrack


Frame t_1
0.9
0.8
0.9
0.1
Frame t_2
0.9
0.4
0.8
0.1


Frame t_1
0.9
0.8
0.9
0.1
Frame t_2
0.9
0.4
0.8
0.1

What ByteTrack Actually Reports

MOT17 test	MOTA	IDF1	HOTA
ByteTrack	80.3	77.3	63.1

at 30 FPS on a single V100.

The claim worth repeating

Applying the same two-stage association to **nine** other state-of-the-art trackers improved their IDF1 by **1 to 10 points**.

A change that helps nine systems you did not write is a change to the problem, not to a model.

And the uncomfortable part

Much of ByteTrack's headline also comes from a very strong detector.

What Breaks: The Camera Moves Too

The constant-velocity model lives in **image** coordinates. Pan or shake the camera and every object in the frame accelerates at once, and nothing in the scene did anything unusual.

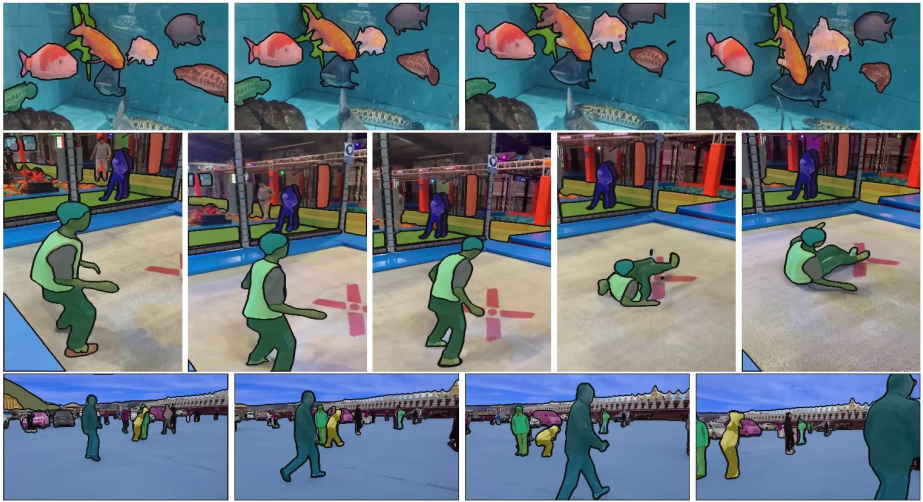
BoT-SORT's fix is to estimate a global **affine** transform between consecutive frames (sparse keypoints, tracked with optical flow, RANSAC) and apply it to the predicted state and covariance *before* association.

MOT17 val	MOTA	IDF1	HOTA
Kalman only	77.67	79.89	68.12
+ camera comp.	78.31	81.51	69.06

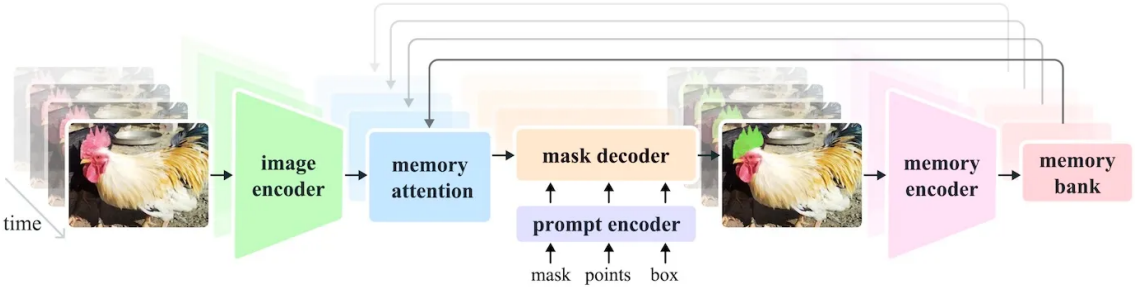
MOT17 test	MOTA	IDF1	HOTA
ByteTrack	80.3	77.3	63.1
BoT-SORT	80.6	79.5	64.6
BoT-SORT+ReID	80.5	80.2	65.0

The rest of the list, in the order you will meet it: identical uniforms, so appearance is noise; occlusions longer than the track buffer; a 5 fps camera, so the IoU gate has nothing to gate on; and objects entering at the frame edge, half-detected.

Tracking Without a Detector: SAM 2 (2024)



SAM 2 (2024)



- Identity is defined by *prompt*, not by a class list. No detector to train, no categories to choose, nothing to label.
- Carries one object at a time — memory is per object, so a crowd costs linearly. It has no notion of "all the people in this frame", so it cannot count, and the MOT metrics still assume a detector.

Use it when you have few objects, no labels, and a human available to click once. Use tracking-by-detection when you have many objects and a number to report.

MOTA Is Broken

Multiple Object Tracking Accuracy

$$\text{MOTA} = 1 - \frac{\sum_t (\text{FN}_t + \text{FP}_t + \text{IDSW}_t)}{\sum_t \text{GT}_t}$$

where Identity Switches (IDSW) count instances where the model successfully detects an object but accidentally swaps its unique tracking ID, and GT is the total number of ground truth objects.

- Dominated by bounding box misses rather than true tracking (association) performance
- In dense or crowded scenes, a tracker's failure to detect a few objects can completely dominate the MOTA score.
- Only considers whether an identity switch happened relative to the previous frame

What To Report Instead

Higher Order Tracking Accuracy (HOTA) (Luiten et al., 2021) scores detection and association separately and multiplies:

$$\mathrm{HOTA}_\alpha = \sqrt{\mathrm{DetA}_\alpha \cdot \mathrm{AssA}_\alpha}$$

averaged over localisation thresholds α from 0.05 to 0.95 — so it also measures how well you localised, which MOTA does not.

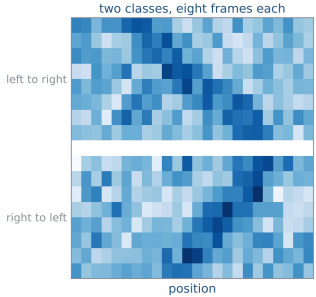
- **Detection Accuracy (DetA)**: Measures how well the tracker finds objects in each frame without creating false positives or missing true objects.
- **Association Accuracy (AssA)**: Measures how consistently the tracker maintains the same unique ID for each object across the entire video sequence

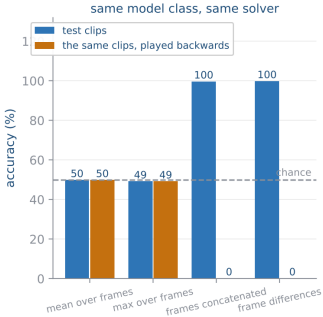
A geometric mean cannot be rescued by one factor, must do both.

Report HOTA with DetA and AssA beside it. Two numbers tell a reader which half of tracker to doubt; one number tells nothing.

Action Recognition

A Frame Classifier Is Not Enough





why pooling cannot work

$$\frac{1}{T}\sum_{t}\phi(x_t)$$

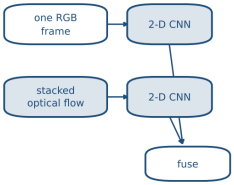
A mean, a max, a sum: every one of them is invariant to the order of its arguments. So a model built on pooled frame features gives the same answer for a clip and its reverse, and the two classes here are exact reverses.

It is not that it is hard. It is impossible, and no amount of data fixes it.

The cheapest video model is a frame classifier plus average pooling, and on many benchmarks it is embarrassingly competitive --- which says more about those benchmarks than about the model. Here the label lives only in the order of the frames, and the symmetry is exact: pooled features cannot separate the classes, concatenation can, and differences do it best. Something-Something was built around this observation, with 174 classes written as templates like "pushing something from left to right".

Three Places to Put Time

two-stream (2014)



motion is an input, not something learned

I3D (2017): inflation

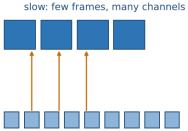
a 2-D kernel



Repeat each 2-D filter N times along time and divide by N. A still image, repeated, then gives exactly the activations it gave before --- so the whole ImageNet model transfers.

Carreira & Zisserman call this "a boring video". Kinetics then lifted UCF-101 from the low 90s to 98.0%.

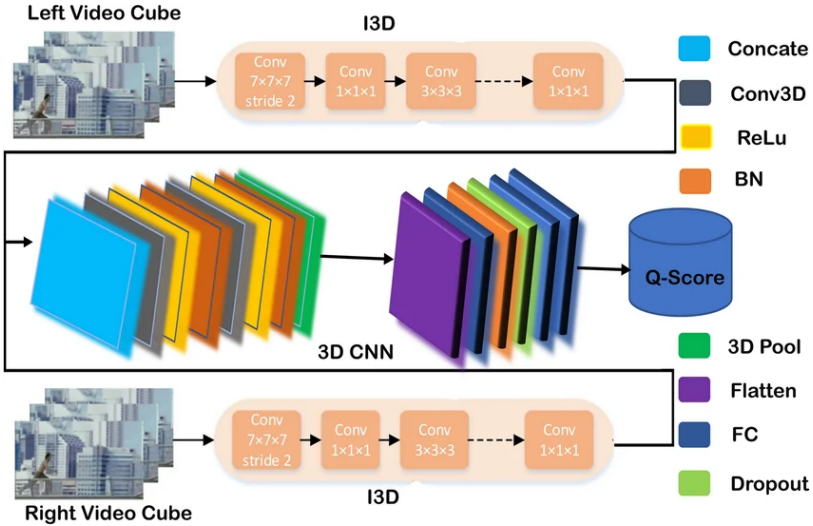
SlowFast (2019)



Two sampling rates instead of one:
semantics are slow, motion is fast.
The fast path is only ~20% of the compute.

Three answers to the same question --- where does time enter the network. Two-stream computes motion outside and feeds it in. I3D builds it into every kernel and inherits ImageNet by a trick. SlowFast says the two things you want from a video change at different rates, so sample them at different rates.

I3D (2017): Inflated 3D CNN



Inflation, and the Boring Video

Take a 2-D architecture that works, give every $N \times N$ filter a time axis to make it $N \times N \times N$, and initialise it from ImageNet by **repeating each 2-D filter N times and dividing by N**.

Feed that network a still image repeated into a video — the authors' "boring video" — and by linearity it produces exactly the activations the 2-D model produced. So the ImageNet weights transfer *exactly*, not approximately.

And then Kinetics

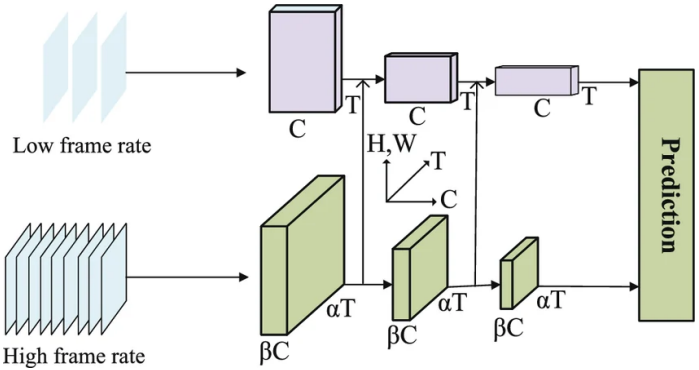
The real contribution was the dataset. After Kinetics pre-training, two-stream I3D reaches 98.0% on UCF-101 and 80.9% on HMDB-51.

On Kinetics itself: 71.1 RGB, 63.4 flow, 74.2 for both, with ImageNet pre-training.

Note what the flow stream is still doing in 2017

RGB alone scores 71.1; adding a hand-computed optical flow stream adds 3.1 points. Three years of 3-D convolutions had not learned what Lucas–Kanade computes in closed form.

SlowFast (2019): Two Rates, Not One

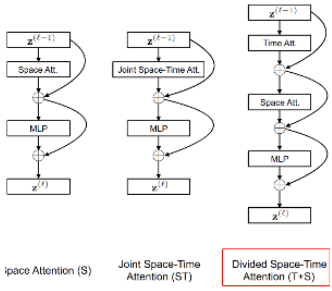


Two pathways: a slow one with few frames and many channels, and a fast one with $\alpha = 8\times$ the frames and $\beta = 1/8$ the channels, with lateral connections feeding fast into slow.

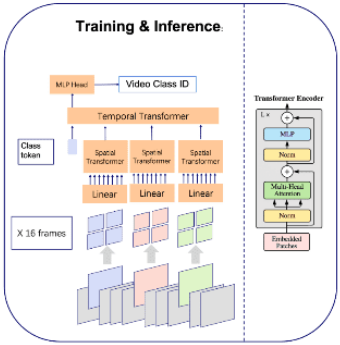
A design principle rather than an architecture: when two signals in your problem have different natural time constants, sampling them at the same rate is a choice, and usually the wrong one.

Video Transformers

Space-time attention in self-attention



Space-time attention as Transformer blocks



Tokens grow linearly with the number of frames and attention grows with their square. A video transformer that attends jointly over space and time is quadratic in clip length.

What Time Costs

Video papers do not classify a video once. The standard protocol takes **10 temporal clips** spread over the video, **3 spatial crops** each, and averages the 30 predictions.

SlowFast 8×8 R50 costs 65.7 GFLOPs per view, so one video-level label costs

$$30 \times 65.7 \;=\; 1{,}971 \text{ GFLOPs.}$$

A ResNet-50 forward pass on one image is 3.8 GFLOPs. That is **about 520 images** of compute, for one label.

And what the extra buys

SlowFast 8×8	GFLOPs/view	top-1
R50	65.7	77.0
R101	106	77.9

1.6× the compute for 0.9 points. And papers compare at 30 views; your deployment will not.

What the 30 views are really admitting

The model sees roughly two seconds at a time, and the label belongs to the whole video. Averaging views is how the field papers over the fact that nobody's architecture spans a minute.

What the Benchmarks Measure

Kinetics rewards the scene

"Playing guitar" is a guitar. "Swimming" is a pool. A great deal of Kinetics is answerable from a single frame, and a frame classifier with average pooling is a stronger baseline than it has any right to be.

That is not a flaw in the models. It is what the label set permits.

Something-Something refuses to

174 classes written as **caption templates** over 100,000+ videos: "pushing something from left to right", "pretending to pick something up".

The object is deliberately irrelevant and the scene tells you nothing. Only the order of the frames does.

How to tell which you are on

Run the time-reversal test from two slides ago on *your* data: train on normal clips, evaluate on reversed ones. If accuracy barely moves, your task does not need video, and you should stop paying for it.

For Your Project

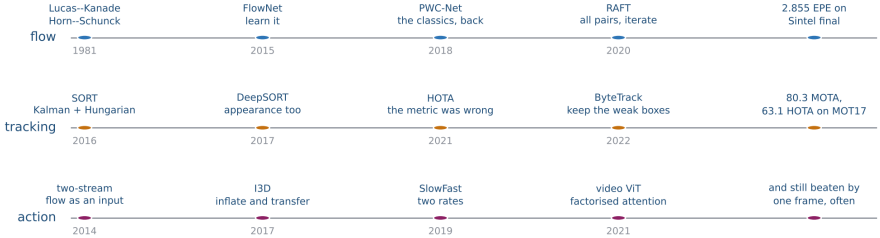
Choosing

If you need	Start with	Note
dense motion	RAFT, pretrained	do not train flow yourself
objects followed through a clip	SORT	then add appearance only if needed
crowded scenes, occlusion	ByteTrack	most of the gain is the detector
a moving camera	BoT-SORT	compensate before you associate
a few objects, no labels	SAM 2, prompted once	no detector, but it cannot count
a clip-level label	a frame classifier + pooling	as the baseline to beat
genuine temporal reasoning	SlowFast or a video ViT	and a dataset that needs it
long video	shot segmentation first	attention is quadratic in length

Before any of that

Check your frame rate. Half of what looks like a hard tracking problem is a 5 fps camera, and half of what looks like a hard action problem is a clip too short to contain the action.

Three Problems, One Shape



Three problems, one shape. Each began with a hand-built method that encoded the right structure, replaced it with a network that learned the structure badly, and then put the structure back inside the network --- a pyramid and a cost volume, a motion model and an exact assignment, two sampling rates. The lesson is the one from Lecture 9: the prior was never the problem, hard-coding it was.

Summary

Flow

- Brightness constancy gives one equation and two unknowns; the structure tensor decides where that bites.
- Linearisation holds for about a pixel. Pyramids, warping and cost volumes exist to get past it.
- RAFT: all pairs, then iterate.

Tracking

- Detection you already have; tracking is association, and association is Lecture 8's assignment problem along t.
- A motion model buys cheap association and survival through occlusion.
- MOTA weights detection $\sim 100\times$ association. Report HOTA, with DetA and AssA.

Action

- Pooling over frames is exactly invariant to time reversal, so it cannot represent order at all.
- Time enters as an input, as a kernel, or as a second sampling rate.
- Attention is quadratic in clip length, so video transformers factorise it.

The thread through all three

Each field hand-built the right structure, abandoned it for a network, then rebuilt it inside the network. The prior was never the problem.

Reading & Reference

Flow

- Horn & Schunck, *Determining Optical Flow*, AI 1981
- Lucas & Kanade, *An Iterative Image Registration Technique*, IJCAI 1981
- Dosovitskiy et al., *FlowNet*, ICCV 2015
- Sun et al., *PWC-Net*, CVPR 2018
- Teed & Deng, *RAFT*, ECCV 2020

Tracking

- Bewley et al., *Simple Online and Realtime Tracking*, ICIP 2016
- Wojke et al., *DeepSORT*, ICIP 2017
- Luiten et al., *HOTA*, IJCV 2021
- Zhang et al., *ByteTrack*, ECCV 2022
- Aharon et al., *BoT-SORT*, arXiv 2022 — camera motion
- Ravi et al., *SAM 2*, ICLR 2025

Action

- Simonyan & Zisserman, *Two-Stream Convolutional Networks*, NIPS 2014
- Carreira & Zisserman, *Quo Vadis, Action Recognition?* (I3D), CVPR 2017
- Goyal et al., *The "something something" Video Database*, ICCV 2017
- Feichtenhofer et al., *SlowFast Networks*, ICCV 2019
- Bertasius et al., *TimeSformer*, ICML 2021

Code

- `torchvision.models.optical_flow` — RAFT, pretrained
- `mmtracking` / `boxmot` — SORT, DeepSORT, ByteTrack
- `TrackEval` — HOTA, and the only implementation to trust
- `pytorchvideo`, `mmaction2` — I3D, SlowFast, ViTs

Hands-On

`lecture11_handson.ipynb`: the three sections, one exercise block each.

1. Brightness constancy from scratch; draw the constraint lines and find where the structure tensor is singular
2. Lucas–Kanade, then a pyramid: measure where the linearisation breaks and how far coarse-to-fine gets you
3. Horn–Schunck: sweep λ and measure the boundary penalty
4. Build a correlation volume and read the three cost surfaces
5. Greedy against Hungarian association; measure the disagreement rate on random crowded frames
6. A constant-velocity Kalman filter through an occlusion
7. Implement MOTA, IDF1, DetA and AssA, and reproduce the pathology from this lecture
8. The time-reversal test: four feature maps, one solver